

Documento de Arquitetura & Design

Especificação Técnica da Landing Page — AluCar

Projeto: AluCar Web Platform

Arquitetura: CSR / Jamstack

Status: Aprovado para Produção

1. Padrão Arquitetural Escolhido

Client-Side Rendering (CSR) & Component-Based Architecture

A aplicação foi estruturada seguindo uma arquitetura moderna orientada a componentes de interface desacoplados, processados diretamente no lado do cliente (*Client-Side*).

Justificativa Técnica:

- **Desempenho e Fluidez:** Roteamento e interações ocorrem sem a necessidade de recarregar a página (*Single Page Application / Landing Page*), proporcionando uma experiência de usuário (UX) imediata e sem fricção.
- **Desacoplamento de Camadas:** A camada de apresentação (Front-end) fica completamente isolada do backend. Caso a AluCar integre uma API REST de reservas no futuro, a interface já está pronta para consumir esses serviços via requisições assíncronas (`fetch` / `axios`).

2. Diagrama de Arquitetura do Sistema

O diagrama visual abaixo detalha a organização estrutural dos módulos da aplicação e a separação clara entre a camada de apresentação e a camada de comportamento:

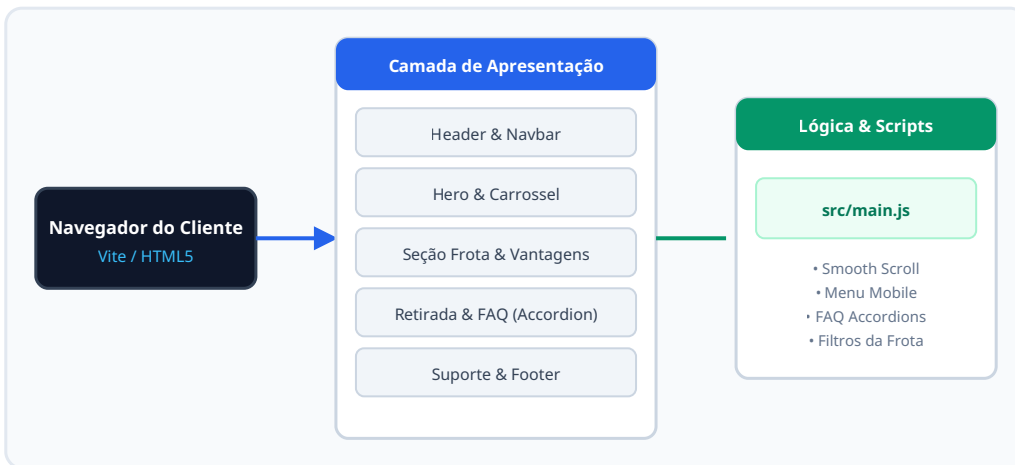


Figura 1: Fluxo de renderização CSR e acoplamento dos scripts com os componentes de UI.

3. Princípios SOLID Aplicados

1. Single Responsibility Principle (SRP) — Princípio da Responsabilidade Única

Aplicação: Cada seção/componente do HTML e cada arquivo do projeto possui uma única responsabilidade no sistema. A seção de `#suporte` cuida estritamente dos canais de atendimento, a seção `#frota` é responsável exclusivamente pela exibição dos veículos, e o arquivo `style.css` lida apenas com as regras globais e Tailwind.

Exemplo: Separação estrita entre a barra de navegação principal (`<nav>`) e o formulário de contato, prevenindo que um componente assuma responsabilidades de layout alheias.

2. Open/Closed Principle (OCP) — Princípio Aberto/Fechado

Aplicação: A estrutura visual criada para os cards de veículos da frota e os pontos de retirada foi desenhada para ser **aberta para extensão, mas fechada para modificação**.

Exemplo: Para adicionar novos carros à frota ou criar filiais, basta inserir novos elementos no grid padronizado via HTML sem a necessidade de refatorar as regras CSS ou o container pai.

4. Design Patterns Utilizados

Pattern 1: Module Pattern (Padrão Módulo)

Descrição: Utilizado na organização do código JavaScript através do ecossistema de ES Modules do Vite (`src/main.js`). O código é totalmente isolado em escopos modulares para impedir a poluição do escopo global (`window`).

Justificativa: Evita colisão de nomes de variáveis, melhora a segurança do estado e aumenta significativamente a manutenibilidade.

Pattern 2: Component / Template Pattern (Componentização Visual)

Descrição: Padronização visual dos elementos reutilizáveis utilizando as classes utilitárias do Tailwind CSS.

Justificativa: Assegura um Design System consistente (cores, espaçamentos, tipografia e estados de `hover` / `active`) em toda a aplicação.

5. Tecnologias Utilizadas e Justificativas

Tecnologia	Função no Projeto	Justificativa Técnica
HTML5 Semantic	Estruturação de conteúdo	Uso de tags semânticas (<code><header></code> , <code><nav></code> , <code><main></code> , <code><section></code> , <code><footer></code>) para maximizar a acessibilidade (a11y) e a indexação nos motores de busca (SEO).
Tailwind CSS v3	Estilização utilitária	Permite prototipagem ágil, eliminação automática de CSS não utilizado no build final (<i>PurgeCSS</i>) e excelente suporte ao design responsivo (<i>Mobile-First</i>).
JavaScript (ES6+)	Interatividade e lógica	Manipulação direta e performática da DOM para controle de navegação, rolagem suave (<i>smooth scroll</i>) e interações dinâmicas (accordion do FAQ).
Vite	Bundler & Dev Server	Desempenho superior de compilação utilizando ES Modules nativos, servimento instantâneo em desenvolvimento (HMR) e otimização automatizada do bundle final.